

【提分教程】赛题四

金融长文本 Agent 的动态记忆压缩与高效问答挑战

本文档面向已经跑通 baseline 流程、希望对系统做进一步优化的参赛者。建议先阅读 Baseline 教程 再回来。

赛事官方地址：<https://tianchi.aliyun.com/competition/entrance/532486>

1. 赛题题目类型分析

A 组 100 道题看似都是“读文档选答案”，实际按解题方法可以分成几类。理解这些类型，才能明白为什么“搜索到片段直接回答”远远不够。

1.1 公式计算题

题目给出具体数值，要求从条款中提取计算公式后代入计算。

示例：ins_a_002（退保金排序）

平安智盈金生累计所交保费 10 万元，保单账户累计收益 2 万元，在第 7 个保单年度末退保；国寿增益宝个人账户价值 12 万元，在第 7 个保单年度末退保（退保费用 0%）；平安富鸿金生现金价值 9 万元。则三者退保所得金额从高到低排序为？

这道题需要从 3 份保险条款中分别找到退保金的计算公式：

- 平安智盈金生：第 6-10 年现金价值 = 累计保费 + 收益 $\times 75\% \rightarrow 10 + 2 \times 0.75 = 11.5$ 万
- 国寿增益宝：退保费用 0%，退保金 = 个人账户价值 = 12 万
- 平安富鸿金生：现金价值直接给出 = 9 万

排序：12 > 11.5 > 9 $\rightarrow$ 选项 A 正确。

这类题的难点是公式分散在不同文档，必须逐份定位条款，提取参数，再做算术。只搜“退保”两个字得到的片段大概率漏掉“第 6-10 年”这个关键条件。

示例：ins_a_003（多保险联合赔付计算）

李某因白血病复发住院，总费用 8 万元，医保报销 3 万元，自费 5 万元。众安白血病医疗险免赔额为 0；平安 e 生保（计划二，免赔额 5000 元）；太保团体百万医疗（免赔额 1 万元）。三家保险公司共应赔付多少？

这道题涉及 3 份医疗险条款，每份的赔付规则不同：有的先扣免赔额再按比例赔，有的 0 免赔全额赔，还要看费用是否属于保险责任。最终答案需要通过多条款交叉计算得出，不是单文档搜索能解决的。

1.2 跨文档对比题

题目明确要求对比两份或多份文档中的数据或条款。

示例：fin_a_001（比亚迪连续两年年报对比）

根据比亚迪连续两年的年度报告，下列关于公司经营业绩变化的描述中，哪些是准确的？

- A. 2025 年营业收入较 2024 年实现增长
- B. 2025 年归属于上市公司股东的净利润出现下滑
- ...

需要分别从 2024 年报和 2025 年报中提取营业收入、净利润、经营现金流、研发投入等数据，再逐项对比增长/下滑关系。单看任意一份年报都不够。

示例：fin_a_005（跨公司财务指标对比）

基于比亚迪与美的集团 2024 年年度报告的数据对比，关于两家企业关键财务指标的结论，哪些是成立的？

这道题需要同时打开比亚迪和美的两份年报，提取同一类指标（营业收入规模、研发投入强度、经营现金流等）做横向比较。如果检索模块只返回了比亚迪的数据，美的的数据没召回，答案就直接错一半。

1.3 多文档联合推理题

题目涉及多个产品/法规/条款，需要把各文档的信息拼起来做综合判断。

示例：ins_a_001（四产品身故保险金排序）

关于四个养老保险/寿险产品的"身故保险金"，以下计算结果正确的是？假设所有产品的已交保费均为 100 万元...平安智盈金生（领取日前）保单账户价值 90 万元；国寿增益宝（一人，40 岁）基本保额 90 万元，个人账户价值 85 万元...

涉及 4 份保险条款，每份的身故保险金计算公式都不一样：有的是"已交保费 vs 现金价值取大"，有的是"已交保费 × 比例 + 账户价值"，有的是"基本保额 × 系数"。必须四份条款都读到、都算对，才能排序。

示例：ins_a_005（多产品免责范围判断）

下列哪些情形属于相应保险产品的免责范围？

- A. 平安智盈金生：被保险人因酒后驾驶导致失能失智护理状态
- B. 平安安佑福重疾险：被保险人因感染艾滋病病毒（非输血、职业、器官移植）导致重大疾病...

4 个选项分别来自 4 份不同条款，每个选项都要到对应条款里找"责任免除"章节，确认该情形是否在免责列表中。这本质上是 4 次独立的条款定位任务，拼在一起成了一道多选题。

1.4 条款定位题

题目问的是某一条款的具体规定，答案就在文档的某个章节里。

示例：ins_a_004（等待期内出险处理）

关于“等待期”内出险的处理，以下哪种情形下保险公司需要承担赔偿责任？

A. 投保平安安佑福重疾险第 50 天，因意外车祸导致双目失明（符合重大疾病）

...

4 个选项涉及 4 份不同保险，每份的“等待期”条款可能都有例外规定（比如“因意外伤害发生上述情形的，无等待期”）。需要逐份定位等待期条款，再判断该选项情形是否适用例外。

1.5 事实查找题

题目问的是文档中直接记载的事实，不需要计算，只需要定位。

示例：res_a_001（行业数据查找）

关于银保渠道发展历程及银行 IT 市场规模的描述，下列哪些选项是正确的？

A. 韩国寿险银保渠道保费贡献率在 2022 年达到了 56%

B. 2005 年至 2018 年期间，银保渠道实现复合增速 9.9%

...

这类题在研报中通常有明确的数字和结论，但研报动辄几万字，数字散落在不同段落。如果检索只返回了前半部分，可能恰好漏掉了“2022 年”这个限定词。

小结

这五种类型不是互斥的，很多题同时涉及多种能力。比如 ins_a_003 既是**多文档联合推理**（3 份条款），又是**公式计算**（每份条款的赔付公式不同）。fin_a_001 既是**跨文档对比**（两年年报），又是**事实查找**（具体数值）。

这意味着一个有效的系统至少要解决两个问题：

1. 如何确保所有相关文档都被读到（跨文档题不能只读一篇）
2. 如何在文档中精确定位关键信息（计算题要找到公式，条款题要找到对应章节）

硬截断策略对第 1 个问题无能为力（4 篇文档每篇截断 4000 字，总长度可能超限），对第 2 个问题取决于关键信息是否恰好在 4000 字内。后续优化方向，本质上都是在解决这两个问题。

1.6 什么是 RAG（检索增强生成）

上面两个问题的标准解决方案，在 NLP 领域有一个专门的名字：**RAG (Retrieval-Augmented Generation, 检索增强生成)**。它的核心思想很简单：在让模型回答之前，先从文档中把相关内容找出来，只把最相关的片段交给模型，而不是把整篇文档都塞过去。

标准的 RAG 流程分三步：

(1) 索引 (Indexing)

把所有文档切成小块 (chunk)，每块几百到几千字符，然后用 Embedding 模型把每块文本编码成向量，存入向量数据库。这样每段文本都有了一个"地址"，可以通过向量相似度快速定位。

(2) 检索 (Retrieval)

收到用户问题后，把问题也编码成向量，到向量数据库里搜索最相似的 top K 个文本块。这一步的本质是"找证据"，即从海量文档中挑出可能包含答案的那几段。

(3) 生成 (Generation)

把检索到的文本块和原始问题一起拼成 prompt，交给 LLM 生成最终答案。因为 LLM 看到的不是整篇文档，而是经过筛选的最相关片段，所以既减少了噪音，又降低了 Token 消耗。

RAG 与本赛题的关系

金融长文本问答本质上就是一个 RAG 任务：给定一个问题（如"平安智盈金生的退保金怎么算"），系统需要从指定文档中**检索**到相关条款，然后**生成**正确答案。本方案的硬截断等于跳过了"检索"这一步，直接把文档前 4000 字全部塞给模型，让模型自己从噪音里找信号。后续优化的核心，就是逐步补齐 RAG 的各个环节：更好的分块策略、更准的检索算法、更合理的片段组装方式。

1.7 典型题目深度解析：reg_a_014

为了更直观地理解赛题的难度和解题方法，下面以一道官方提供的典型题目为例，展示完整的证据定位、逻辑推理和答案推导过程。

题目概况

字段	内容
qid	reg_a_014
领域	regulatory（监管法规）
题型	多选题（multi）
关联文档	strict_csrc_023《上市公司治理准则》、 strict_csrc_035《上市公司章程指引》

题干：

某 A 股上市公司（非金融类，总股本 5 亿股，最近一期经审计净资产 120 亿元）拟进行四项决策：

- ① 为资产负债率为 75% 的全资子公司提供 8 亿元担保；
- ② 将原募投项目“智能工厂建设”剩余募集资金 6.5 亿元变更为“新能源材料研发中心”；
- ③ 选举新一届独立董事，其中 1 名候选人系某律所合伙人，其所在律所过去 12 个月内曾为公司提供常年法律顾问服务但未承办重大诉讼；
- ④ 审议《关于修订公司章程第 42 条股东大会特别决议事项的议案》，拟将“对外提供财务资助”纳入须经特别决议的范围（原章程未列明）。

选项：

- **A：**事项①中，因被担保方资产负债率超过 70%，且担保金额占公司最近一期经审计净资产比例达 6.67%，两项任一条件触发即须经股东大会审议通过，故该项担保必须提交股东大会审议。
- **B：**事项②中，变更募集资金用途涉及金额 6.5 亿元，虽未达净资产 10%（12 亿元），但因变更投向属于“主营业务方向重大调整”，根据《章程指引》第 44 条及《治理准则》第 19 条，仍须经股东大会特别决议通过。
- **C：**事项③中，该独立董事候选人存在《治理准则》规定的“最近 12 个月内为上市公司提供过法律服务”的情形，构成独立性障碍，不得提名为独立董事候选人，故股东大会不得对其行使选举权。
- **D：**事项④中，公司章程自主增设特别决议事项不违反《章程指引》第 77 条“上市公司可在本指引基础上增加特别决议事项”的授权性规定，但须经股东大会普通决议通过即可生效，无需特别决议。

标准答案：AC

证据定位与推理过程

选项	关键法条定位	推理
A	《章程指引》第四十七条第（四）款："为资产负债率超过百分之七十的担保对象提供的担保"须经股东会审议通过	被担保方资产负债率 75% > 70%，触发法定审议条件，A 正确
B	《章程指引》第八十一条（普通决议事项）、第八十二条（特别决议事项）	变更募集资金用途属于普通决议事项，未列入特别决议清单，B 错误
C	《治理准则》第四十一条：独立董事不得与上市公司存在"直接或间接利害关系"	候选人所在律所过去 12 个月提供过法律服务，构成利害关系，C 正确
D	《章程指引》第八十二条第（三）款："本章程的修改"须经特别决议通过	将"对外财务资助"纳入特别决议范围属于修改章程，必须经特别决议，D 错误

这道题对系统的考验：

1. **跨文档定位**：需要同时在《章程指引》和《治理准则》中查找对应条款，不能只读一份文档
2. **精确条款匹配**：每个选项的推理都依赖具体条款编号（第四十七条、第八十二条），模型必须定位到精确条款而非泛泛而谈
3. **条件判断**：选项 A 中的"资产负债率超过 70%"是一个明确的数值阈值，模型必须准确提取并比较
4. **逻辑否定**：选项 B 和 D 都是"看起来合理但实际错误"的干扰项，模型需要识别出"变更募集资金用途是普通决议""修改章程是特别决议"这些否定性结论
5. **多选题陷阱**：漏选 C 或多选 B/D 都不得分，必须四个选项全部判断准确

这道题的难度在于：它不是"找到原文就能答对"的简单题，而是需要**精准定位多个法条**→**提取关键条件**→**逐项逻辑判断**→**综合得出多选组合**的复杂推理链。如果检索模块漏掉了《章程指引》第八十一条和第八十二条的区分，或者模型把"普通决议"和"特别决议"搞混，答案就直接错了。

2. PDF 解析方案

RAG 系统的质量上限由两个因素决定：检索策略和文档解析质量。Baseline 教程第七章已经指出了 PDF 解析质量是第一个瓶颈，但只给了方向没有展开。这一章详细对比主流方案，并解释 baseline 的选择逻辑。

2.1 为什么 PDF 解析是第一步且最重要

整个 pipeline 的链路是：PDF → 解析 → 文本 → 切片 → 检索 → LLM 回答。如果第一步就把表格变成了乱码、公式丢失了、段落顺序乱了，后面所有环节都在错误的文本上工作。这种错误无法通过优化检索或 prompt 来补救。

保险条款和财报类 PDF 有几个让解析器头疼的特点：

- **复杂表格**：保费表、现金价值表、赔付比例表，行列密集且嵌套层级深
- **数学公式**：精算公式、财务比率公式，上下标和分式结构容易丢失
- **双栏/多栏布局**：部分研报和合同使用多栏排版，阅读顺序容易错
- **页眉页脚混入正文**：页码、章节标题、公司名称等重复内容污染正文
- **扫描件 vs 电子件**：部分文档是扫描图片（需要 OCR），部分是原生 PDF（可直接提取文字），两者需要不同的处理策略

2.2 主流方案对比

（1）轻量级：PyMuPDF4LLM（当前 baseline 方案）

PyMuPDF4LLM 是基于 PyMuPDF（原 fitz）的轻量封装，专门为 LLM 场景设计。它的核心能力是把 PDF 直接转为 Markdown 文本，保留标题层级（通过字号和字体推断），提取表格为 Markdown table 格式。

优势：

- 纯 Python 库，安装简单（`pip install pymupdf4llm`）
- 不需要 GPU，纯 CPU 运行
- 速度快，一份 50 页 PDF 约 5-10 秒完成
- 对原生 PDF（非扫描件）的文字提取质量尚可
- 输出 Markdown 格式，直接兼容下游处理

局限：

- 不包含 OCR 能力，扫描件 PDF 无法处理
- 表格提取基于规则，复杂嵌套表格容易错位

- 公式识别能力弱，数学公式可能丢失结构
- 双栏文档的阅读顺序可能出错
- 字号推断标题层级不总是准确（比如正文小标题和普通段落字号相同）

为什么 baseline 选它： 赛方评测环境不提供 GPU，PyMuPDF4LLM 是当时能在 CPU 上快速跑通的方案。17% 的准确率中，有一部分损失确实来自 PDF 解析不够好，但 baseline 的目标是先把端到端流程跑通。

(2) 中量级：Marker

Marker (VikParuchuri, 2024, 持续更新至 2025 年 1.4.0 版本, GitHub: VikParuchuri/marker) [1] 是一个基于 pipeline 架构的 PDF 转 Markdown 工具。它的流水线是：Surya 布局检测 → Surya OCR（可选）→ 文本提取 → Markdown 输出。

核心能力：

- 布局检测使用 Surya 模型，能识别标题、正文、表格、公式、图片、页眉页脚等区域
- 内置 OCR 能力，支持 90+ 种语言
- 对扫描件和原生 PDF 都能处理
- 输出格式丰富：Markdown、JSON（含位置信息）
- 支持 GPU 加速，也能纯 CPU 运行（速度较慢）

优势：

- 表格和公式处理优于纯规则方案
- 多语言支持好，中文文档识别质量不错
- 自动过滤页眉页脚
- 社区活跃，更新频繁

局限：

- 依赖 PyTorch 模型，需要 4-8GB 显存（GPU 模式）
- CPU 模式下速度较慢，一份 50 页 PDF 可能需要 5-15 分钟
- 对极端复杂的表格（多层嵌套、合并单元格）仍有遗漏
- 公式识别不支持 LaTeX 输出，公式以图片或文本形式保留

对本赛题的适用性： 如果赛方提供 GPU 环境，Marker 是一个性价比不错的选择。CPU 模式下虽然慢，但解析在预处理阶段完成，不占用提交时的 Token 预算，时间成本可以接受。

(3) 重量级：MinerU

MinerU（上海 AI Lab / OpenDataLab, 2024 年发布, 持续更新至 2026 年 4 月 3.1.0 版本, GitHub: opendatalab/MinerU) [2] 是目前开源社区最受关注的 PDF 解析工具之一, 专为 LLM / RAG / Agent 场景设计。

核心能力:

- 完整的版面分析管线: 布局检测 → 阅读顺序恢复 → 元素识别 → 结构化输出
- 表格和公式提取质量高, 公式自动转为 LaTeX
- 支持 109 种 OCR 语言
- 输出 Markdown / JSON / HTML / LaTeX 多种格式
- 内置 MCP Server 支持, 可直接集成到 Agent 系统
- 兼容 LangChain、LlamaIndex、Dify、RAGFlow 等主流 RAG 框架

优势:

- 公式转 LaTeX 是本赛题的关键优势——保险条款和财报中的数学公式能完整保留
- 阅读顺序恢复能力强, 双栏/多栏文档不乱序
- 对中文文档优化好 (上海 AI Lab 团队中文背景)
- OmniDocBench 评测中英文文档解析表现领先 [3]
- 同时支持本地部署、在线 API、CLI 和 Python SDK

局限:

- GPU 依赖: 需要 CUDA 支持的 GPU (推荐 8GB+ 显存), 纯 CPU 模式可以运行但非常慢
- 安装复杂: 依赖较多系统库, 首次配置可能需要 30 分钟以上
- 内存占用大: 处理高分辨率文档时可能占用 10GB+ 内存
- 单页处理时间: GPU 下约 3-8 秒/页, CPU 下可能 30-60 秒/页

对本赛题的适用性: 如果赛方允许使用 GPU 环境, MinerU 是目前的最佳选择。公式 LaTeX 化、表格结构化、阅读顺序恢复这三项能力直接解决了本赛题 PDF 解析的核心痛点。即使赛方只给 CPU 环境, MinerU 也可以跑 (只是慢), 预处理阶段的时间成本通常可以接受。建议关注赛方最终公布的环境配置。

补充说明: Meta 在 2023 年发布的 Nougat 是最早用端到端 Transformer 做 PDF 公式解析的工作之一, MinerU 的公式处理模块 UniMERNet 吸收了其思路并做了改进。Nougat 作为独立工具已不再活跃维护, 且专为英文学术论文设计, 不适用于本赛题的金融文档, 此处不再展开。

(4) 视觉语言模型级：olmOCR

olmOCR (Allen AI, 2025 年 2 月发布, arXiv:2502.18443) [4] 代表了 PDF 解析的新方向：不用传统的 OCR + 版面分析流水线，而是用一个精调的 7B 视觉语言模型 (VLM) 直接把 PDF 页面转成结构化的纯文本。

核心能力：

- 基于 7B VLM，精调在 260,000 页多样化 PDF 上
- 支持表格、列表、公式、手写文字的提取
- 输出线性化的纯文本，保持自然阅读顺序
- 批处理效率高，100 万页仅需 176 美元（使用开源模型时几乎免费）
- 在 olmOCR-Bench 上表现优于 GPT-4o、Gemini Flash 2 和 Qwen-2.5-VL

优势：

- 不需要复杂的版面分析管线，一个模型搞定所有页面类型
- 对扫描件、手写文档、低质量 PDF 的处理能力远超传统 OCR 方案
- 开源的 7B 模型可以在本地部署，不需要调用付费 API

局限：

- 需要 GPU (7B VLM 推理)，CPU 不可用
- 中文文档的处理质量未知（训练数据以英文为主）
- 输出格式为纯文本而非 Markdown，丢失了部分结构信息
- 表格和公式以文本形式呈现，不如 LaTeX 精确

对本赛题的适用性： 如果赛方提供 GPU 环境且中文处理质量过关，olmOCR 是一个值得尝试的方案。但目前中文支持情况不明朗，建议等社区有中文评测结果后再决定。

2.3 本赛题的 PDF 解析选型建议

根据赛方可能提供的环境，分三种情况：

环境	推荐方案	理由
纯 CPU	PyMuPDF4LLM + 预处理优化	无 GPU 时的唯一选择。可以在预处理阶段做更多工作来补偿解析质量（如人工修正表格、补充公式描述）
GPU（8GB+）	MinerU	当前开源方案中综合能力最强，公式/表格/阅读顺序三项核心能力突出
GPU（16GB+）	MinerU + olmOCR 互补	MinerU 处理结构化文档，olmOCR 处理扫描件和低质量页面，两路互补

即使赛方最终只给 CPU 环境，学习 MinerU 和 olmOCR 也是有价值的。了解更好的解析方案能帮助理解当前 baseline 的瓶颈在哪里（哪些错误是 PDF 解析导致的，哪些是检索策略导致的，哪些是 prompt 设计导致的）。定位清楚瓶颈，才能精准优化。

2.4 预处理阶段可以做的工作

无论用哪个解析工具，以下预处理手段对提升文本质量有帮助：

1. **页眉页脚过滤**：用正则匹配重复出现的页码、章节标题、公司名称等，统一去除
2. **表格后处理**：对解析出的表格做格式校验（行列数一致、数值列对齐），异常时回退到手动提取
3. **公式补标注**：在公式附近插入描述性文本（如"该公式表示：现金价值 = 累计保费 × 75%"），帮助 LLM 理解公式含义
4. **段落合并**：同一段落的文本可能被解析成多行，按标点和语义做合并
5. **文档结构标注**：为每个段落标注所属章节路径（如"第二章 > 第三节 > 现金价值计算"），增强检索时的上下文信息

3. RAG 检索优化

Baseline 的硬截断方案本质上跳过了检索，直接把文档前半部分塞给模型。要突破 17% 的准确率天花板，第一步就是引入检索模块，但必须注意检索引入的连贯性断裂风险。

3.1 分块策略

文档切片是 RAG 系统的地基。切得不好，后面的检索和生成都受影响。

固定大小分块：按固定字符数或 token 数切分，实现简单但容易切断语义。比如"因意外伤害发生上述情形的，无等待期"这句话如果被切成两块，前半句在一块的末尾、后半句在另一块的开头，检索可能只命中其中一块，模型看到的就不完整。

滑动窗口分块：相邻块之间有重叠（通常 10%-20%），减少边界信息丢失。适合保险条款这种"前后文紧密关联"的文档。重叠太多会导致检索结果重复，占用 token 预算。

语义分块：按段落、章节等自然边界切分，保持每个块内的语义完整。Markdown 文档天然适合这种策略（按标题层级切分）。缺点是块大小不均匀，有的块可能过长。

多粒度分块 (Small-to-Big / Parent-Child)：这是更进阶的策略。把文档切成两层：

- **子块 (Child Chunks)：**较小 (200-500 token)，用于向量检索。小块嵌入更聚焦，检索精度更高。
- **父块 (Parent Chunks)：**较大 (800-2000 token)，包含子块的完整上下文。检索命中子块后，把对应的父块也一并取出来送给 LLM。

这种做法的好处是"检索用小块（精确）、回答用大块（完整）"。实验表明，子块检索 + 父块上下文的方案在准确率上比单一大块方案高 10%-15%。

推荐起步方案：按 Markdown 的 ## 标题切分 + 滑动窗口，块大小控制在 800-1200 token，重叠 15%。有精力的话，升级到多粒度分块（子块 400 token 检索 + 父块 1200 token 回答）。

3.2 混合检索

单一检索策略各有盲区：

- BM25 对精确匹配敏感（数字、条款编号），但无法理解"退保费用"和"退保手续费"是同一个东西
- 向量检索 (Embedding) 能捕捉语义相似性，但对数字和年份不够敏感

混合检索 = BM25 + 向量检索，两路同时召回，结果融合后一起送进 LLM。融合方式：

- **简单合并去重：**两路结果拼接，去重后按原始文档顺序排列
- **RRF (Reciprocal Rank Fusion)：**按每路结果的排名位置加权融合，不依赖绝

对分数。公式： $\text{score}(d) = \sum 1/(k + \text{rank}_i(d))$ ，k 通常取 60

- **加权分数融合**：两路结果归一化后按权重加和（如 BM25 权重 0.3 + 向量 0.7），需要调参

实现要点：

- BM25 用 `rank-bm25` 库，分词用 `jieba`
- 向量检索可选 `text-embedding-3-small`（百炼提供）或本地模型 `bge-large-zh`
- 两路各召回 20-30 个片段，融合后取 top 10-15

BGE 模型系列速览（扩展学习）

注：经向赛题方确认，*Embedding 模型和 Rerank 模型不允许在正式提交中使用。以下内容仅供扩展学习，帮助理解 RAG 系统的完整技术栈，不建议在正式方案中依赖。*

BAAI（北京智源人工智能研究院）开源的 BGE 系列是目前中文 RAG 场景最常用的模型族，覆盖了 Embedding 和 Reranker 两类需求。了解各型号的差异，有助于根据实际算力条件选型：

Embedding 模型（向量检索）：

模型	参数量	特点	适用场景
bge-large-zh-v1.5	326M	中文专用，C-MTEB 榜单前列	纯中文文档，CPU 可跑但推荐 GPU
bge-m3	568M	多语言，支持 8192 token 长文本，可同时输出 dense + sparse 向量	中英混合文档，或需要长上下文嵌入
bge-multilingual-gemma2	9B	最新一代，MTEB 多语言榜首，精度最高	GPU 充足时的最强选择

Reranker 模型（重排序）：

模型	参数量	特点	适用场景
bge-reranker-base	278M	轻量，CPU 可跑，中文效果不错	算力受限时的默认选择
bge-reranker-large	560M	精度高于 base，需要 GPU	有 GPU 时的升级选择
bge-reranker-v2-m3	568M	多语言，与 bge-m3 配套使用	中英混合文档的重排序

对本赛题的建议：如果赛方给 GPU，**bge-m3**（Embedding）+ **bge-reranker-v2-m3**（Reranker）是多语言场景的稳妥组合；如果只有 CPU，**bge-large-zh-v1.5** + **bge-reranker-base** 可以在可接受的速度下完成检索和重排。

视觉检索：跳过 OCR 的另一种思路

前面的所有方案都遵循同一个范式：PDF → 解析为文本 → 切片 → 文本检索。但如果 PDF 解析本身就是瓶颈（表格乱码、公式丢失、双栏错序），有没有可能直接跳过解析？

ColPali (2024) 和 **ColQwen** (2025) [15] 提出了视觉检索的思路：把 PDF 页面渲染成图像，用视觉语言模型（VLM）直接从页面图像中提取 patch 级别的嵌入，然后基于这些视觉嵌入做检索。整个过程不需要 OCR、不需要版面分析、不需要 Markdown 转换。

具体做法：

1. 把每页 PDF 渲染为高分辨率图像（如 448×448）
2. VLM 对图像做 patch 编码（如 ColPali 产出每页 1024 个 patch embedding）
3. 查询时，问题文本和页面图像的 patch embedding 做 MaxSim 匹配（取每个 query token 与所有 patch 的最大相似度之和）
4. 返回最相关的页面

优势：

- 完全绕过了 PDF 解析的所有问题：表格、公式、双栏都不需要处理
- 布局信息天然保留（表格的视觉结构、公式的排版位置）
- ViDoRe 评测基准上，ColPali 的检索精度超越了传统 OCR + 文本检索方案

局限：

- 需要 VLM (3B+ 参数)，GPU 必须
- 返回粒度是整页，不是精确的文本片段：模型需要自己从页面图像中“读”出答案
- 目前以英文文档为主，中文文档的视觉检索效果待验证
- ColModernVBERT 提供了一个 250M 参数的轻量替代，精度接近 ColPali 但模型小 10 倍

对本赛题的启示：视觉检索提供了一个激进的思路：如果 MinerU 或 Marker 的解析效果在特定文档上仍然不理想，可以直接把页面当图片检索，搭配多模态 LLM（如 GPT-4o、Qwen-VL）来“看图答题”。这在保险条款的复杂表格和公式场景下可能有奇效。但 GPU 要求和中文适配是两个现实障碍，建议作为实验性方向而非主方案。

3.3 重排序

初步检索的结果中可能混入弱相关片段。重排序用更精细的模型对候选片段逐一打分，把真正相关的排到前面。

常见方案：

- **Cross-Encoder 模型**：把"问题 + 候选片段"拼接后送入模型，直接输出相关性分数。BGE Reranker 系列的具体型号选择见上节速览表。
- **LLM-based Rerank**：用 LLM 自己对候选片段打分。优点是灵活（可以按自定义标准评分），缺点是慢且消耗 token。

建议：如果使用百炼 Embedding API，可以搭配百炼的 Rerank API；如果本地部署，见上节 BGE 速览表按算力条件选择。

3.4 查询增强

查询增强策略（如 HyDE、查询改写、查询分解）旨在缩小用户口语化提问与文档专业术语之间的语义差距。具体方案见 3.6 节"前沿 RAG 范式"中的 HyDE 讨论。

3.5 片段组装

检索到的片段不能直接拼接就送给 LLM。需要注意：

- **去重**：滑动窗口会导致相邻片段内容重叠，需要去重
- **排序**：按文档原始顺序排列片段（而不是按相关性分数），保持原文的逻辑流
- **截断**：总 prompt 长度受 token 预算限制，需要在片段数量和质量之间做取舍

3.6 前沿 RAG 范式

前面讲的分块、混合检索、重排序都属于"标准 RAG"的优化手段，在分块质量、检索精度、排序准确性上做改进。近一年来，RAG 领域出现了几个重要的新范式，它们不是调参数，而是改变了检索的底层结构：从"搜碎片"升级到"理解结构"。

理解这些范式的关键，在于搞清楚它们各自在解决什么问题。传统 RAG 有三个根本缺陷：

1. **扁平化**：所有文档块在同一层，检索只看块级别的相似度，无法理解跨块的语义结构（一个公式分散在三段里，单块检索无法还原）
2. **局部性**：擅长回答"这段话说了什么"，但回答不了"这些文档整体讲了什么"（四份保险条款的退保规则有什么系统性差异？）
3. **无反思**：检索到什么就用什么，不评估质量，不修正方向

下面的范式各自针对其中一个或多个缺陷。

GraphRAG：用知识图谱做检索

GraphRAG (Microsoft Research, 2024 年 4 月发布, arXiv:2404.16130) [5] 的核心思路：不要只把文档切成碎片去搜，而是先从文档中提取实体和关系，构建知识图谱，再基于图谱做检索和推理。

传统 RAG 本质上在做"向量空间中的邻居搜索"：把问题和文档块都映射到向量空间，找距离最近的块。这种做法的上限是"找相似的文本片段"，但无法回答需要跨片段推理的问题。比如"这四份保险条款中，哪份对意外身故的赔付最高？"，这个问题需要先分别理解四份条款的身故保险金计算规则，再横向比较，而不是找到某个"相似"的片段就够了。

GraphRAG 的完整流程：

- 1. 实体与关系抽取**：用 LLM 从源文档中提取实体和关系。实体可以是产品名 ("平安智盈金生")、概念 ("现金价值")、数值 ("75%")、条款编号 ("第 7 保单年度")；关系描述实体之间的逻辑连接 ("现金价值 $\rightarrow$ 计算公式 $\rightarrow$ 累计保费 $\times$ 75%"、"意外身故 $\rightarrow$ 适用规则 $\rightarrow$ 无等待期")。
- 2. 知识图谱构建**：将抽取出的实体作为节点、关系作为边，构建一张有向/无向图。这张图不是简单的关键词共现网络，而是携带了语义信息的知识结构。
- 3. 社区检测**：用 Leiden 算法在图结构上发现紧密关联的实体群组 (社区)。每个社区对应一个主题领域，比如"退保规则社区"可能包含"退保费用""退保费用率""现金价值""保单年度"等实体及其关系。
- 4. 社区摘要生成**：对每个检测出的社区，用 LLM 生成一个总结性的摘要，描述这个社区的核心信息。这一步是 GraphRAG 区别于传统图方法的关键：不是把原始文本片段直接存到图里，而是对每个主题生成高层摘要，使得检索时可以直接读取"关于退保规则的核心要点是什么"。
- 5. 两种检索模式**
 - Local Search (局部搜索)**：查询时匹配相关实体，从实体出发沿图的边扩散到邻居节点，获取与问题直接相关的实体和关系。适合事实查找类问题 ("平安智盈金生的现金价值怎么算？")。
 - Global Search (全局搜索)**：查询时匹配相关社区，用社区的摘要生成部分回答，再将所有部分回答汇总成最终答案。适合全局理解类问题 ("四份条款的退保规则有什么系统性差异？")。这一步本质上是 Map-Reduce：社区摘要做 Map，LLM 汇总做 Reduce。

对本赛题的适用性：

跨文档对比题和多文档联合推理题是 GraphRAG 的最优场景。比如 ins_a_001 (四产品身故保险金排序)，传统 RAG 需要逐份文档检索每份产品的身故保险金规则，再在

prompt 里手动对比。GraphRAG 可以在知识图谱中直接查询"身故保险金"这个实体，沿关系边找到四个产品的计算规则，甚至社区摘要已经预先生成了"四产品身故保险金对比"的总结。

代价是预处理阶段需要大量 LLM 调用：实体抽取（每份文档）、社区摘要（每个社区）。对于本赛题的 100 题对应的文档集，这个开销是可控的，大约几十到几百次 LLM 调用，可以在提交前离线完成，不占用正式提交的 Token 预算。

实现参考： 微软开源了 [graphrag](#) Python 包，支持本地运行。对于本赛题的中文金融文档，需要替换内置的实体抽取 prompt 为中文版本。

LightRAG：轻量级图 RAG

LightRAG（香港大学 HKUDS 实验室, 2024 年 10 月发布, EMNLP 2025, arXiv:2410.05779）[6] 是 GraphRAG 的轻量级替代方案，保留了图结构的优势，但大幅降低了构建成本和检索延迟。

与 Microsoft GraphRAG 的关键差异：

GraphRAG 的社区检测 + 逐社区摘要是一个重操作：Leiden 算法需要遍历整张图，每个社区的摘要又需要独立的 LLM 调用。LightRAG 不依赖社区检测，而是用**双向向量索引**来同时支持局部和全局检索：

1. **实体向量索引：** 每个实体节点有独立的向量表示（由实体名称和描述的 embedding 组成）。查询时，问题向量与实体向量做相似度匹配，找到相关实体后沿关系边扩散到邻居节点。这对应 GraphRAG 的 Local Search。
2. **关系向量索引：** 每条关系边也有独立的向量表示（由关系描述和两端实体信息的 embedding 组成）。查询时，问题向量与关系向量做相似度匹配，直接找到相关的关系。这能捕捉到跨实体的语义模式（比如"退保费用率逐年递减"这种趋势性关系），而不仅仅是单个实体的匹配。
3. **关键词引导查询：** LightRAG 从用户问题中提取关键词，将这些关键词的向量拼接成组合查询向量。这种方式比直接用整个问题做向量检索更聚焦，减少了语义稀释。

增量更新能力：

LightRAG 的图结构支持增量插入——新文档加入时，只需要提取新文档的实体和关系、计算它们的向量、插入图结构中，不需要重建已有索引。这对本赛题可能很重要：A 榜只有 100 题对应的一批文档，如果 B 榜下发新文档，LightRAG 可以直接增量更新，而 GraphRAG 需要重新跑社区检测。

对本赛题的定位：

LightRAG 适合作为从标准 RAG 向图 RAG 过渡的第一个实验方案。它的构建成本接近传统 RAG（主要开销在实体抽取，不需要社区检测），但能同时支持局部事实查找和全局对比推理。如果本赛题后续出现需要跨文档联合推理的难题，LightRAG 的性价

比优于全量 GraphRAG。

实现参考：开源项目 [HKUDS/LightRAG](#) 提供了完整的 Python 实现。

StructRAG：按任务类型选择知识结构

StructRAG（2024 年发布）[7] 提出了一个有意思的视角：不同任务需要不同的知识组织方式，不应该统一处理。

它定义了五种结构类型，根据任务特征自动选择最合适的结构：

结构类型	适用任务	组织方式
表格 (Table)	统计类任务	结构化行列
图 (Graph)	长链推理任务	实体-关系网络
算法 (Algorithm)	规划类任务	步骤流程
目录 (Catalog)	总结类任务	层级分类
片段 (Chunk)	简单单跳问答	原始文本块

对本赛题的启示：不同题型应该用不同的知识组织方式。事实查找题用片段检索就够了；计算题需要从文档中提取公式构建成“算法”结构；跨文档对比题需要图结构。一个统一的检索框架可能不如按题型分派检索策略来得有效。

RAPTOR：树状递归摘要检索

RAPTOR (Recursive Abstractive Processing for Tree-Organized Retrieval)

(2024 年 1 月发布, arXiv:2401.18059) [8] 解决的是"扁平检索丢失全局上下文"的问题，思路和 GraphRAG 不同：GraphRAG 用图结构，RAPTOR 用树结构。

为什么需要树结构：

传统 RAG 把所有文档块平铺在一层。检索时只看单个块的相似度，但有些问题的答案跨越了多个块。一个典型的保险条款场景：某产品的退保规则可能分布在"第二章 现金价值"（定义公式）、"第五章 退保"（退保流程）、"附录"（费率表）三个不同章节。这三个章节在文档中相距很远，但在语义上紧密关联。扁平检索很难同时召回这三个章节的片段。

RAPTOR 的构建流程：

1. **叶子层**：将文档切成基础块（如每页 Markdown），每个块做 embedding
2. **聚类**：用高斯混合模型（GMM）对叶子节点的 embedding 做软聚类，每个节点可以属于多个聚类
3. **摘要**：对每个聚类，用 LLM 对聚类内的所有文本块生成一个摘要。这个摘要成为上一层的一个节点
4. **递归**：对上层节点重复聚类 + 摘要，直到只剩一个根节点
5. **最终结构**：一棵多层树，底层是原始文本块（精确细节），中层是主题摘要（如"退保规则""身故保险金规则"），顶层是全局摘要（如"该产品的核心条款概述"）

检索时的层级选择：

- 简单事实查找（"平安智盈金生第 7 年的现金价值是多少？"）→ 从叶子层检索，直接拿精确数值
- 中等复杂度（"该产品的退保规则有哪些注意事项？"）→ 从中层摘要节点检索，获取主题级别的完整信息
- 全局理解（"该产品的整体保障结构是怎样的？"）→ 从顶层摘要节点检索

变体与改进：

- **adRAP** (Adaptive RAPTOR)：解决 RAPTOR 在动态数据集上的问题。原始 RAPTOR 每次新增文档都需要重建整棵树（因为聚类会变）。adRAP 通过增量调整聚类结构来避免全量重建，适合文档持续增加的场景。
- **postQFRAP**：不改变检索端的索引结构，而是在检索结果回来之后，对召回的片段做一次"查询聚焦的递归摘要处理"：把检索结果按 RAPTOR 的思路聚类+摘要，生成一个结构化的上下文再送给 LLM。优势是可以作为后处理模块接入任何检索管线。

对本赛题的适用场景：

行业研报类题目（如 res_a_001 银保渠道发展历程）和跨年度对比题（如 fin_a_001 比亚迪两年年报对比）是 RAPTOR 的优势场景：这些题需要同时理解宏观趋势和微观数据，树结构天然匹配"总-分"的信息粒度。

Self-RAG 与 CRAG：让模型反思检索质量

传统 RAG 无条件信任检索结果：搜到什么就用什么，不管质量如何。这在实践中是个严重问题：BM25 可能因为关键词匹配把不相关的片段排到前面，向量检索可能因为语义模糊召回弱相关的内容。Self-RAG 和 CRAG 在这里加了"自我评估"机制。

Self-RAG（2024 年 10 月发布, arXiv:2310.11511）[9] 的核心是"检索 + 反思 + 重试"循环：

1. 按需检索：不是每个问题都需要检索。Self-RAG 先判断当前问题是否需要检索（简单常识问题可能不需要）
2. 检索文档片段
3. 对每个片段做三方面评估：相关性（这个片段和问题相关吗？）、支持度（这个片段支持还是反对某个观点？）、有用性（这个片段对回答有帮助吗？）
4. 对低质量片段重新检索或修正
5. 生成最终答案时，标注每个论断的来源片段和可信度

CRAG (Corrective RAG)（2024 年 1 月发布, arXiv:2401.15884）[10] 在 Self-RAG 基础上更进一步，加入了自动纠错机制：

1. 检索文档片段
2. 用检索质量评估器对整体检索结果打分（0-1 分）
3. 根据分数分三级处理：
 - 分数 ≥ 0.7 (CORRECT)：检索质量好，直接使用
 - 分数 0.3-0.7 (AMBIGUOUS)：质量一般，保留当前结果同时用不同关键词补充检索，两轮结果合并
 - 分数 ≤ 0.3 (INCORRECT)：质量差，丢弃检索结果，切换到全文阅读或其他策略
4. 最终生成前再做一次"幻觉检查"，过滤掉检索结果中不存在的论断

CRAG 的关键洞察：检索质量的评估不需要复杂的模型。用检索结果中每个片段与问题的关键词重叠度、片段间的语义一致性等轻量特征，就能大致判断检索质量。只有当评分落在中间区域时才需要 LLM 介入做进一步判断。

对本赛题的适配：

本赛题不允许网络搜索，但 CRAG 的三级处理思路可以直接适配：

- CORRECT → 直接用检索结果生成答案
- AMBIGUOUS → 换关键词重新检索同一文档集，或者扩大检索范围（如从只搜前 10 个块扩展到搜全部块）
- INCORRECT → 放弃检索，改为读全文（回到 baseline 的硬截断方案作为 fallback）

这个机制的价值在于：它让系统在“检索结果好”的时候享受检索的效率，在“检索结果差”的时候自动回退到更稳健的方案，而不是一味信任检索。

HyDE：用假设答案做检索

HyDE (Hypothetical Document Embeddings) (2023 年 12 月发布, arXiv:2212.10496) [11] 解决的是“用户问题用词和文档用词不一致”导致检索失败的问题。

问题场景：用户问“怎么退保最划算”，文档里写的是“现金价值计算方式”和“退保费用率表”。问题中的“划算”是口语化表达，文档中的“现金价值”是专业术语，两者在向量空间中距离很远，直接向量检索可能召回到完全不相关的内容。

HyDE 的做法：先让 LLM 凭空生成一份假想的“理想答案”，再把这份假想答案（而非原始问题）转成向量去检索。假想答案是用文档风格写的（包含“现金价值”“退保费用”“已交保费”等专业术语），所以更容易匹配到真实的相关片段。

流程：用户问题 → LLM 生成假设答案 → 假设答案 Embedding → 向量检索 → 返回真实片段。

代价是每次检索多一次 LLM 调用（生成假设答案），但检索命中率提升显著：在多个 benchmark 上，HyDE 的召回率比直接向量检索高 10%-20%。对本赛题的口语化问题（如“哪个产品最划算”）特别有效。

HyDE + 混合检索的实践建议：根据“Searching for Best Practices in RAG”（2024 年 7 月发布, arXiv:2407.01219）[12] 的系统评测，**HyDE + 混合检索 (BM25 + 向量)** 是目前综合表现最好的默认配置。该评测在多个 benchmark 上对比了不同检索策略，HyDE + Hybrid Search 的检索质量显著优于单独的查询改写 (Query Rewriting) 或查询分解 (Query Decomposition)。

KAG：专业领域知识增强

KAG (Knowledge Augmented Generation) (2024 年发布) [14] 针对的是专业领域 RAG 的一个特殊问题：通用知识图谱的实体和关系定义与专业领域不匹配。

例如，用通用 OpenIE 工具从保险条款中抽取实体，可能把“现金价值”标注为普通名词而非专业概念，把“第 7 保单年度”拆成两个无关实体。KAG 的做法是引入**领域知识对**

齐：用领域概念体系（如保险术语表）来指导实体抽取和关系定义，减少信息抽取过程中的噪音。

对本赛题的启示：如果使用 GraphRAG/LightRAG 做实体抽取，需要定制实体类型（产品名称、条款编号、计算公式、数值、条件判断）和关系类型（属于、计算依据、例外条件、时间约束），而不是用通用 prompt。否则抽取出的实体关系图可能充满噪音，反而干扰检索。

多跳推理 RAG (Multi-hop RAG)

多跳推理是指答案不能从单个文档片段直接获得，需要通过多个片段的推理链来推导。本赛题中这类问题很常见："某产品在第 N 年的退保金额是多少？"需要先找到退保公式（第一跳），再找到该年度的退保费用率（第二跳），再找到该年度的现金价值（第三跳），最后代入计算。

传统 RAG 对多跳问题的处理很弱：检索可能只召回第一跳的片段（退保公式），但漏掉了后两跳（费用率和现金价值）。针对多跳 RAG 的改进方案：

- **Iterative RAG (迭代检索)**：第一轮检索 → 生成部分答案 → 根据部分答案生成新查询 → 第二轮检索 → 汇总。每轮检索聚焦一个子问题
- **IRCoT (Interleaving Retrieval with Chain-of-Thought)**：把思维链推理和检索交替进行。每推一步就检索一次，用检索结果来验证和修正推理方向
- **Beam Search RAG**：对多跳路径做束搜索，同时探索多条推理链，最后选证据最充分的一条

Agentic RAG：让 Agent 自主编排检索策略

前面所有的 RAG 范式都预设了检索策略：什么时候搜、搜什么、搜几次，都是提前写死的。Agentic RAG 的思路是把这个决策权交给 LLM 自己：通过 Function Calling, LLM 自主决定：

- 当前问题需要检索吗？（简单常识题可能不需要）
- 搜什么关键词？（可能需要多轮尝试不同关键词）
- 搜到了什么程度才算足够？（自主判断信息是否足够）
- 搜到的东西质量差怎么办？（自动切换策略）

这和本赛题的 Agent 定位高度一致。Baseline 测试过 Function Calling, Token 消耗比 no-tool 高约 18%，但 FC 的灵活性（模型可以自主决定"什么时候搜、搜什么"）是提升准确率的关键路径。具体方案见第六章"Agent 化方向"。

3.7 本赛题的 RAG 路线建议

从 baseline 的硬截断出发，建议按以下顺序逐步引入 RAG 优化：

阶段	方案	预期收益	风险
1	BM25 检索 + 语义分块	解决"答案不在前 4000 字"的问题	连贯性断裂，可能降分
2	+ 多粒度分块（子块检索 + 父块回答）	缓解连贯性断裂	实现复杂度增加
3	+ 混合检索（BM25 + 向量）+ HyDE	提升召回率，改善口语化问题匹配	需要 Embedding 模型，HyDE 增加一次 LLM 调用
4	+ Rerank 精排	提升精确率	额外计算开销
5	+ 动态记忆压缩（Snip + Memento + 结构化摘要）	在有限 Token 预算内保留最多有效信息，多文档场景收益明显	压缩策略调优需要实验，过度压缩可能丢失关键条件
6	+ Bash 工具（Python 代码执行做数值计算）	消灭公式计算题的算术错误	需要设计沙箱执行环境
7	+ CRAG 自检机制（质量评估 + 自动回退）	减少低质量检索导致的错误	需要设计评估器逻辑
8	LightRAG（跨文档题）	解决跨文档联合推理，支持增量更新	预处理 Token 消耗
9	Agentic RAG（FC 自主编排检索策略）	灵活应对不同题型	Token 消耗增加约 18%，需要调优 FC 行为
10	smolagents Code Agent（Python 代码编排工具调用）	减少 30% 推理步骤，精准数值计算	需要学习 smolagents 框架，代码生成质量依赖模型能力

建议每完成一个阶段就跑一次 A 榜提交，验证实际收益，避免"加了优化反而降分"的陷阱。

4. 上下文管理与动态记忆压缩

赛题名称里就有"动态记忆压缩"，这其实定义了本赛题的核心技术挑战。前面三章（PDF 解析、RAG 检索）解决的是"怎么找到有用信息"，这一章解决的是"怎么在有限 Token 预算内把最重要的信息喂给模型"。

4.1 问题的本质：上下文窗口是稀缺资源

每道题的推理链路可以拆成：读文档 → 提取信息 → 判断 → 输出答案。每一步都消耗 Token，但预算有限（赛方总预算 500 万 Token）。这意味着不能把所有文档全读一遍，也不能把检索结果全塞进 prompt。

上下文管理的核心问题可以表述为：**给定 Token 预算 B，如何从候选信息池中选择一个子集，使得模型回答正确的概率最大化？**

候选信息池包括：

- 文档原文片段（检索结果）
- 上一步推理的中间结果
- 题目本身和选项
- Prompt 模板
- 验证/自检的二次推理结果

4.2 动态截断

Baseline 的硬截断固定取前 4000 字符。更好的策略是让模型自己决定什么时候停止阅读。

实现思路：分段读取文档，每读一段就让模型判断"当前信息是否足够回答问题"。如果足够就停止，不够就继续读下一段。这本质上是 Agent 化的第一步。

更精细的做法是**按题型设定不同的阅读预算**。事实查找题可能读前 2000 字符就能找到答案；跨文档对比题需要读多份文档，每份可能 1000 字符就够；公式计算题只需要定位到公式所在的段落，读到公式后就可以停。

4.3 信息密度控制

保险条款和监管法规中有大量模板化文本（如"本合同的构成""投保须知"等），这些内容对答题几乎没有价值，但占用了大量 token。

可以预处理时过滤掉高频无意义段落，或者在检索时对"释义""名词定义"等章节降权。更系统化的做法是**按题型定义信息价值函数**：对计算题，公式段落的权重最高；对条款定位题，例外条件段落的权重最高。

4.4 动态记忆压缩：分层存储与选择性遗忘

这才是本赛题"动态记忆压缩"的核心。前面的截断和密度控制都是静态策略，动态压缩需要在推理过程中持续做决策：什么该记住、什么该压缩、什么该丢弃。

工业界的实践：Agent 的记忆分层

几个主流 Agent 框架都有类似的记忆管理机制，核心思路都是**分层存储**：

Kimi Code CLI 的 Context Compaction [19]

当对话历史超过 token 阈值时自动触发 compaction，让 LLM 将历史对话压缩为结构化摘要，按优先级保留：

1. **Current Task State**（当前任务状态：正在做什么）
2. **Errors & Solutions**（遇到的错误和解决方案）
3. **Code Evolution**（代码最终版本，丢弃中间尝试）
4. **System Context**（项目结构、环境配置）
5. **Design Decisions**（架构决策及理由）
6. **TODO Items**（未完成的任务）

Compaction 前后有 PreCompact / PostCompact hooks 可插拔，允许用户自定义压缩策略（如阻止特定信息的压缩）。压缩后的历史替换原始对话，模型只看到摘要。

这个机制对本赛题的启发：压缩的核心是有选择地保留、改写、丢弃。Kimi Code 的 6 级优先级给出了一个信息价值排序的框架。对于本赛题，我们可以定义类似的优先级：

1. 题目的完整文本和所有选项（最高优先级，绝不压缩）
2. 从文档中提取出的公式、数值、条件判断（核心证据）
3. 检索到的原文片段中与题目直接相关的部分
4. 文档章节标题和结构信息（用于定位和引用）
5. 中间推理过程的摘要
6. 检索到的原文中经判断为不相关的部分（最低优先级，优先丢弃）

Hermes 的 Trajectory Compressor [20]

Hermes 是面向通用 Agent 任务的框架，其 `trajectory_compressor.py` 实现了一套完整的对话历史压缩管线，虽然是为训练数据压缩设计的，但核心算法可以直接迁移到推理场景。

压缩算法的完整流程：

第一步：计算压缩需求。 遍历 trajectory 中每个 turn，用 tokenizer 计数。如果总 tokens 未超过目标上限（默认 15250），直接跳过压缩。

第二步：确定受保护区域。 在 trajectory 中标记不可压缩的 turns：

- 头部保护：第一个 system 消息、第一个 human 消息、第一个 gpt 回复、第一个 tool 调用。这些是任务的初始上下文，丢弃后模型会丢失任务定义。
- 尾部保护：最后 N 个 turns（默认 N=4）。这些包含最新的推理状态和即将做出的决策。

第三步：计算压缩区域。 受保护头部之后、受保护尾部之前的所有 turns 构成“可压缩区域”。从这个区域的起始位置开始，逐 turn 累积 tokens，直到累积量满足：

Plaintext

累计 tokens $\geq$ (总 tokens - 目标上限) + 摘要目标 tokens

如果遍历完整个可压缩区域仍不满足，则压缩整个区域。

第四步：LLM 生成摘要。 将被压缩的 turns 拼接为一段文本，发送给单独的摘要模型（默认 gemini-3-flash，temperature=0.3）。摘要 prompt 明确要求包含：

- 助手执行了哪些操作（工具调用、搜索、文件操作）
- 获得了什么关键信息或结果
- 做出了什么重要决策或发现
- 相关的数据、文件名、数值、输出

摘要以 [CONTEXT SUMMARY]：为前缀，目标长度 750 tokens。

第五步：组装压缩后的 trajectory。 头部 turns（原文保留）+ 摘要消息（作为 human 消息插入）+ 尾部 turns（原文保留）。如果 system 消息中配置了 `add_summary_notice`，会在 system prompt 末尾追加提示文字，告知模型部分历史已被压缩。

压缩效果度量： 每条 trajectory 压缩后记录详细 metrics：

- `compression_ratio`: 压缩后 tokens / 原始 tokens
- `tokens_saved`: 节省的 tokens 数

- `turns_removed`: 被移除的 turns 数
- `still_over_limit`: 压缩后是否仍超预算

对本赛题的关键启发：

Hermes 压缩器给了一个精确的算法框架：在 **token 预算约束下**，保护首尾、压缩中间、**LLM 替代**。迁移到本赛题的单题推理场景：

- **"首"是什么？** 题目文本 + 选项（不可压缩，必须完整保留）
- **"尾"是什么？** 最终推理步骤（列出公式 → 代入数值 → 计算结果 → 对比选项 → 输出答案）。这一段的推理链必须完整保留，因为任何压缩都可能导致逻辑跳跃
- **"中间"是什么？** 文档阅读过程。比如读了一份 50 页的保险条款，中间可能读了 20 页才找到"现金价值"的定义。这 20 页的阅读过程可以压缩为一段结构化摘要："该产品的现金价值计算规则：第 1-5 年为已交保费 × 100%，第 6-10 年为已交保费 × 75%，第 10 年后为已交保费 × 50%。退保费用率为逐年递减，第 7 年起为 0%。"
- **摘要替代：**用 LLM（或同一个小模型）生成文档阅读摘要，替代原始阅读内容。摘要的关键要求是：**结构化、数值精确、标注出处（页码/段落号）**。

Claude Code 的四层渐进压缩 [21]

Claude Code 把压缩拆成四层，从便宜到贵逐级尝试：

Plaintext

Layer 1: Snip（裁剪工具输出）→ 免费，不调 LLM

Layer 2: Microcompact（基于 prompt cache 的轻量压缩）

Layer 3: Context Collapse（折叠搜索/读取操作为摘要）

Layer 4: Autocompact（LLM 全量总结）→ 最贵

核心设计原则：只有低层搞不定才升级。Snip 只是把老工具结果截断为头尾各保留一部分，不消耗 token；Autocompact 要调一次 LLM 生成全量摘要。如果 80% 的场景 Snip 就够了，只有 20% 需要 LLM 介入，整体 token 效率远高于每次都调 LLM。

对本赛题的启发：检索返回的文档片段如果太长，先 Snip（只保留包含关键词的段落 + 前后各一段上下文）；Snip 后仍然超预算再调 LLM 做摘要。不要一上来就让模型总结。

Codex 的 Memento 策略：为什么不能只留摘要 [22]

Codex 的 compaction 有一个关键设计：压缩后的历史不是纯摘要，而是

`initial_context` + 精选 user 消息 + `summary` + 最近 N 条。原因很简单，LLM 生成的 `summary` 会丢失精确信息。

一个本赛题中的典型场景：保险条款里"因意外伤害发生上述情形的，无等待期"这句例外条件，压缩时可能被总结为"等待期内一般不赔"，"意外伤害例外"这个关键信息就丢

了。如果后续推理依赖这个例外条件判断选项对错，摘要导致的错误不可逆。

Codex 的做法是：user 原始消息尽量保留（最多 20k tokens），只在实在放不下时才把最早的 user 消息纳入摘要。摘要负责"大局观"（任务进展、关键决策），原始消息负责"精确性"（具体数值、条件判断、例外条款）。

对本赛题来说，这意味着压缩时至少要保留三类信息不被摘要化：

1. 题目原文（包括选项）：压缩题目等于修改题目
2. 从文档中提取的精确数值和公式：如"现金价值 = 累计保费 × 75%"
3. 包含例外条件的条款原文：如"但意外伤害导致的情形除外"

generic-agent 的 L1-L4 记忆层级 [24]

这是最系统的分层方案，来自一个面向长期自主运行的 Agent 项目。虽然不是专门为本赛题设计的，但其分层哲学和信息分类决策树可以直接借用。

四层架构的完整定义：

L1: 全局内存索引 (global_mem_insight.txt)

硬约束 ≤30 行、期望 <1k tokens。内容只包含：

- 场景关键词 → 记忆定位映射：高频场景直接给出 SOP 文件名或 L2 节名；低频场景仅列关键词，需要时再 read L2 或 L3
- RULES (避坑准则)：压缩版的红线规则和高频犯错点。每条不超过一句话，如"搜索用 google 不用百度""禁无条件杀 python（会杀自己）"

L1 的更新纪律极严：L2/L3 有新增/删除时才更新 L1；修改时不允许 overwrite 或 code run，只能少量 patch。不写 How-to、不写技术细节、不写日志。

L2: 全局事实库 (global_mem.txt)

存储环境特异性事实：IP、非标路径、凭证、配置常量等。按 ## [SECTION] 组织。这些是 LLM 零样本无法推理出来的信息。禁止存储易变状态、猜测、或通用常识。

L3: 任务级精简记录库 (../memory/)

为特定任务保留的极简 SOP 和工具脚本。只记录跨会话仍重要、且难以通过少量探测快速重建的要点：

- 该任务特有的隐藏前置条件
- 典型易踩坑点（一旦遗忘会导致高成本重试的信息）
- 高复用的处理脚本（封装复杂逻辑，避免每次重新推理）

不记录普通操作步骤或可在几步探测中重新获得的路径信息。

L4: 历史会话层 (L4_raw_sessions/)

由调度器自动收集的原始对话记录，用于回溯过往上下文。这一层不主动维护，只在需要时被检索。

信息分类决策树：

Plaintext

"这条信息该放哪层？"

是环境特异性事实（IP、凭证、ID 等）？

└ YES → L2，然后按频率归入 L1

└ NO → 是通用操作规律（全局避坑准则）？

└ YES → L1 RULES（仅限 1 句压缩准则）

└ NO → 是特定任务技术（艰难尝试才成功，未来还能用）？

└ YES → L3（专项 SOP 或脚本）

└ NO → 丢弃（判定为通用常识或冗余信息）

核心公理：

- 行动验证原则：**任何写入 L1/L2/L3 的信息，必须源自成功的工具调用结果。严禁将模型的固有知识、推理猜测、未执行的计划作为事实写入。口号：No Execution, No Memory。
- 神圣不可删改性：**凡是经过行动验证的有效配置、避坑指南、关键路径，在重构时严禁丢弃。可以压缩文字、可以迁移层级（从 L2 移到 L3），但不能丢失信息的准确性和可追溯性。
- 禁止存储易变状态：**当前时间戳、临时 Session ID、运行中的 PID、某个具体绝对路径。这些随时间/会话高频变化的数据严禁写入记忆。
- 最小充分指针：**上层只留能定位下层的最短标识，多一词即冗余。

对本赛题的启发：

L1-L4 的分层哲学可以直接映射到单题推理的信息管理：

- L1 等价物：**题目的结构化索引，"这道题属于公式计算题，涉及文档 A 的现金价值公式和文档 B 的退保费用表"。这是一句话的摘要，帮助模型快速定位需要精读的文档段落。
- L2 等价物：**从文档中提取的"硬事实"：公式、数值表、条款编号、条件判断。这些是 LLM 无法凭空生成的，必须从文档中精确提取。
- L3 等价物：**针对特定题型的推理 SOP，比如"计算题 SOP：1) 定位公式 → 2) 提取参数 → 3) 代入数值 → 4) 计算结果 → 5) 匹配选项"。每类题型对应一套 SOP。
- L4 等价物：**完整的文档原文（不压缩，保留在外部存储中，只在需要精确引用时

调取)。

"行动验证原则"在本赛题中的对应是：只有从文档原文中精确匹配到的信息才能作为答案依据。模型的推理猜测 ("看起来像是 75%") 不能作为证据，必须有原文引用。

"最小充分指针"的对应：在对比多份文档时，带着每份文档的结构化摘要 ("产品 X 的身故保险金 = max(已交保费, 现金价值)")，摘要中标注原文出处 (页码/段落号)，需要验证时再回溯原文，不需要带着完整原文。

学术界的方案：MemGPT 与记忆管理研究

MemGPT (UC Berkeley, 2023) [16] 提出了一个根本性的视角转换：给 LLM 一个分层的记忆系统，让它自己管理"什么在窗口内、什么在窗口外"，而不是试图扩展上下文窗口本身。

MemGPT 的核心类比是操作系统的虚拟内存管理：

- **主上下文 (Main Context)**：LLM 的固定上下文窗口，类比物理 RAM。容量有限 (如 4K/8K/32K tokens)，但访问速度最快。LLM 在这里做推理。
- **外部上下文 (External Context)**：几乎无限的长期存储，类比磁盘。存储完整的对话历史、文档、用户信息等。访问速度慢 (需要检索)，但容量几乎不受限制。

MemGPT 的关键创新在于：LLM 自己控制"换页"过程。传统做法是开发者写死截断策略 (取最后 N 轮对话)，MemGPT 的做法是让 LLM 通过 Function Calling 主动管理记忆：

- 当主上下文快满时，LLM 可以调用 `conversation_search` 或 `archival_memory_search` 来检索需要的信息
- LLM 自己判断当前上下文中哪些信息不再需要，将其"换出"到外部存储
- LLM 自己判断下一步推理需要哪些外部信息，将其"换入"主上下文

这本质上是把记忆管理从"系统层的固定规则"升级为"应用层的智能决策"。

MemGPT 的记忆类型：

1. **会话记忆 (Conversational Memory)**：对话历史的滚动窗口。超出窗口的历史被自动分页存储，LLM 通过 `conversation_search` 函数按日期或关键词检索。
2. **档案记忆 (Archival Memory)**：文档、文章、知识库等外部资料。LLM 通过 `archival_memory_search` 函数做语义检索。这部分最接近本赛题的"文档检索"场景。
3. **工作记忆 (Working Memory)**：当前推理状态。LLM 可以通过 `core_memory_append` 和 `core_memory_replace` 函数主动更新 (比如"用户刚才说了喜欢 Python，我把这个记下来")。

后续发展：

- **MemoryOS** [25] 在 MemGPT 基础上扩展为完整的记忆操作系统，引入记忆的热度机制（频繁访问的信息自动提升到更快的存储层）、用户画像的持续更新、以及记忆的主动遗忘策略（基于艾宾浩斯遗忘曲线）。
- **A-Mem (Agentic Memory)** [26] 进一步引入了结构化笔记和知识网络：LLM 动态生成结构化的笔记，并在笔记之间建立链接，形成一个可演化的知识网络，而不是简单地把信息存为文本块。

对本赛题的迁移：

MemGPT 的主上下文 / 外部上下文双层模型可以直接映射到本赛题的推理架构：

MemGPT 概念	本赛题等价物
主上下文	当前题目的 prompt（题目 + 检索结果 + 推理过程），受单题 Token 预算约束
外部上下文	文档全文（已解析为 Markdown 的完整内容），不受单题 Token 预算约束
换页决策	模型判断当前检索结果是否足够回答问题，不够则从文档中检索更多内容
会话记忆	前一道题的推理结果（如果有跨题关联）
档案记忆	文档的 BM25/向量索引

关键差异：本赛题是"长文档 + 短推理"，MemGPT 是"短文档 + 长对话"。所以 MemGPT 的"换页"逻辑在本赛题中应该调整为"分段读取 + 渐进压缩"：把读过的文档段落压缩后换出，释放上下文空间给下一个段落。

具体来说，读一份 50 页的保险条款时：

1. 读第 1-5 页 → 提取关键信息（产品名称、保险期间、基本保额）→ 压缩为结构化摘要，存入"外部上下文"的等价物（Python 变量或临时文件）
2. 读第 6-10 页 → 提取退保规则 → 追加到摘要
3. 读第 11-15 页 → 发现是模板化条款（释义、投保须知）→ 跳过，不消耗主上下文
4. ...继续，直到模型判断信息足够回答问题
5. 最终提交给 LLM 的是结构化摘要而非 50 页原文

对本赛题的迁移

上述所有方案都针对"长对话"场景，但本赛题的场景是"长文档 + 短推理"：每道题是一轮独立的推理，不需要跨题记忆。动态压缩在本赛题中应该用在**单题推理链的内部**：

1. **文档阅读阶段**：分段读取，每读一段就压缩提取关键信息（公式、数值、条件判断），只保留压缩后的摘要进入下一段
2. **多文档对比阶段**：读完文档 A 后生成结构化摘要（如"产品 X：身故保险金 = $\max(\text{已交保费}, \text{现金价值})$ "），读文档 B 时只带着 A 的摘要而非原文，对比时用摘要即可
3. **验证阶段**：自检时不需要重新读全文，只需要带上第一轮推理的结构化摘要（公式 + 代入值 + 计算结果）做交叉验证

具体实现可以参考 Hermes 的 trajectory compressor 思路：保护题目的完整文本（head），对中间文档阅读过程做压缩（middle），保留最终推理步骤的完整链（tail）。

4.5 实现建议：从策略到代码

上面讨论的方案要落地到本赛题，需要回答三个具体问题：什么时候触发压缩、哪些内容保留、摘要用什么格式。

触发策略

参考 Kimi CLI 的双条件触发（`tokens >= 上限 × 比例` 或 `tokens + 预留 >= 上限`），对本赛题的建议：

Plaintext

需要压缩 = (当前 prompt tokens $\geq$ 模型上下文窗口 $\times 0.7$)
或 (当前 tokens + 预计下一轮检索返回的 tokens $\geq$ 模型上下文窗口)

0.7 这个阈值是 Claude Code [21] 和 CheetahClaws [23] 的共同选择：给后续推理留 30% 余量。对于本赛题的单题推理场景，Qwen-plus 从 Qwen-Plus-2025-07-28 快照版本起支持 1M 上下文（默认 128K，可通过 `max_input_token` 参数调整）[27]，即使按默认 128K 计算，约在 90K tokens 时触发第一次压缩。考虑到单篇年报解析后约 5-8 万字符（折合 4-6 万 tokens），读两篇年报就可能触发。

保留策略（Memento 模式）

按优先级从高到低保留：

1. **System prompt + 题目原文**：不压缩，始终保留
2. **从文档中提取的精确数值和公式**：保留原文，不压缩
3. **包含例外条件的条款原文**（如"但...除外""不适用...情形"）：保留原文

4. 中间推理步骤：可压缩为结构化摘要

5. 已读但未用到的文档段落：直接丢弃

具体实现：每次读完一段文档后，先让模型做一个"提取"操作：把这一段里的公式、数值、例外条件摘出来，只保留提取结果进入下一段。这样读 10 段文档可能只需要保留 500 tokens 的提取结果，而不是 5000 tokens 的原文。

结构化摘要模板

压缩时不要用 plain text 总结，用结构化模板。参考 Hermes 的格式：

```
Plaintext
[CONTEXT SUMMARY]
Goal: 判断四款产品的身故保险金高低
Documents Read: doc_1 (平安智盈金生)、doc_2 (国寿增益宝)
Key Formulas Extracted:
- 平安智盈金生: 身故保险金 = max(已交保费, 现金价值)
- 国寿增益宝: 身故保险金 = 基本保额 × 年龄系数
Numerical Values:
- 平安: 已交保费=100 万, 现金价值=90 万 → max = 100 万
- 国寿: 基本保额=90 万, 系数=1.6 → 144 万
Pending Verification: 文档 3 和文档 4 尚未读取
```

这种格式的好处是：后续推理可以直接定位到需要的字段，不需要在摘要里重新搜索。而且结构化的 Key Formulas 和 Numerical Values 不会被 LLM 的总结语言"改写"导致精度丢失。

5. 答案验证与多轮推理

5.1 自检机制

Baseline 单轮提问、单轮回答，没有验证环节。可以加入自检：

1. 第一轮：让模型根据上下文生成答案
2. 第二轮：让模型验证自己的答案，检查每个选项的证据是否充分
3. 如果不一致，回退到第一轮重新生成

5.2 常见错误模式

从实验数据中总结的典型错误：

- 否定词遗漏：文档说"不适用"，模型读成了"适用"
- 年份混淆：题目问 2025 年数据，模型用了 2024 年的
- 条件遗漏：只读到"免赔额 5000 元"，漏掉了"意外伤害 0 免赔"
- 跨文档混淆：把文档 A 的公式套到了文档 B 的场景上

可以在 prompt 中针对这些错误模式加入显式检查指令。

6. Prompt 工程

6.1 题型差异化 Prompt

不同题型需要不同的引导策略：

- 计算题：要求模型先列出公式、再代入数值、再计算结果
- 条款定位题：要求模型先定位条款原文、再逐条比对
- 多选题：要求模型逐选项判断，最后汇总

6.2 Few-shot 示例

在 prompt 中提供 1-2 个已答对的题目作为示例，帮助模型理解答题格式和推理过程。
注意示例不要占用太多 token。

7. Agent 化方向

7.1 Function Calling 方案

Baseline 测试过 Function Calling, Token 消耗比 no-tool 高约 18%。但 FC 的优势在于模型可以自主决策"什么时候搜、搜什么"。

适合 FC 的场景：跨文档题（需要逐份文档搜索）、信息不足时需要补搜的场景。

7.2 多步推理

从"读文档 → 答问题"升级为"读文档 → 提取关键信息 → 判断信息是否足够 → 不够则继续搜索 → 综合推理 → 输出答案"的多步流程。

7.3 终端 Agent 与 Bash 工具

前面讨论的 Agent 大多停留在"读文档 + 调 LLM"的层面。但在本赛题中，有一类题目单纯靠 LLM 的文本推理能力是处理不好的：**公式计算题**。

LLM 的计算短板

大语言模型本质上是"文字接龙"引擎，不是计算器。它在做数学运算时其实是在"背诵"训练数据里见过的计算结果，而不是真正执行算术。举例：

- 题目要求计算 $(8 \text{ 亿} \times 6.67\% + 12 \text{ 亿} \times 75\%) / (1 - 30\%)$
- LLM 直接推理：可能在某一步算错中间结果，或者搞混运算顺序
- 结果：选项 A 和选项 B 的数值差 0.3%，LLM 选错了

这不是模型不够聪明，而是**概率生成天然不适合精确计算**。研究 (ARTIST, 2025) 表明，即使是最强的 LLM 在多步数值计算中的错误率也显著高于使用 Python 解释器辅助的方案。

解决方案：让 LLM 写代码而不是算答案

核心思路：不让 LLM 自己算，而是让它写一段 Python 代码，用 Bash 工具执行代码，再把执行结果还给 LLM 做最终判断。

这是终端 Agent (Terminal Agent / Coding Agent) 模式在金融 QA 中的典型应用。参考 Claude Code (Anthropic, 2025) 和 SWE-bench 中的 Agent 架构，它们共同的特点是：

1. 模型不直接输出答案，而是输出可执行的代码
2. 通过 Bash 工具执行代码，获取精确的数值结果
3. 模型拿到结果后再做语义层面的判断

本赛题中的具体应用

公式计算题（如 ins_a_001 四产品身故保险金排序、ins_a_002 退保金排序）：

模型推理："国寿增益宝的身故保险金 = 基本保额 × 系数，系数取决于被保险人年龄。40 岁对应系数为 1.6。"

模型生成代码：

```
Python
# 国寿增益宝
basic_amount = 900000 # 基本保额 90 万
coefficient = 1.6      # 40 岁对应系数
guoshou = basic_amount * coefficient

# 平安智盈金生
total_premium = 1000000 # 已交保费 100 万
account_value = 900000  # 保单账户价值 90 万
pingan_zhiying = max(total_premium, account_value)

# 排序
results = sorted([
    ("国寿增益宝", guoshou),
    ("平安智盈金生", pingan_zhiying),
], key=lambda x: x[1], reverse=True)
print(results)
```

Bash 执行代码 → 输出精确结果 → 模型判断哪个选项匹配。

跨文档数值对比题（如 fin_a_001 比亚迪两年年报对比）：

模型提取数据："2024 年营收 6023 亿，2025 年营收 7231 亿"

模型生成代码:

```
Python
revenue_2024 = 6023
revenue_2025 = 7231
growth = (revenue_2025 - revenue_2024) / revenue_2024 * 100
print(f"增长率: {growth:.1f}%")
```

Bash 执行 → 增长率: 20.1% → 模型判断"是否实现增长"。

为什么 Bash 工具比 Function Calling 更适合计算

Function Calling 模式通常是这样: 模型调用

`calculator(expression="8*0.0667")`, 然后拿到返回值。这可以处理简单运算, 但有两个局限:

1. **不能写多步逻辑**: 复杂公式需要中间变量、条件判断、循环, 一个 `calculator()` 函数不够用
2. **不能做数据整理**: 跨文档对比需要先把多个文档的数据提取出来, 再统一计算。

Bash 执行的 Python 脚本可以自定义数据结构

Bash 工具的本质是**给模型一个完整的编程环境**, 而不是几个预定义的函数。这让模型可以自己决定"先算哪个、后算哪个、中间结果怎么存、最终怎么排序"。

参考架构：ReAct 循环 + Bash 工具

ReAct (Reasoning + Acting, Yao et al., 2023) 是目前最主流的 Agent 推理范式，也是 Claude Code、OpenHands 等终端 Agent 的基础。

ReAct 的循环是：思考 (Thought) → 行动 (Action) → 观察 (Observation) → 思考 → ...

在本赛题中：

Plaintext

Thought: "这道题需要计算三份保险的退保金。
先从文档中提取各产品的退保金公式和参数。"

Action: 搜索文档，提取公式

Observation: "国寿增益宝：退保金 = 个人账户价值 × (1-退保费用率)
退保费用率第 7 年为 0%"

Thought: "公式和参数已提取，现在用 Python 计算。"

Action: Bash 执行 Python 代码

Observation: "国寿增益宝 = 120000, 平安智盈金生 = 115000,
平安富鸿金生 = 90000"

Thought: "排序：120000 > 115000 > 90000,
对应选项 A: 国寿 > 智盈 > 富鸿。"

这个循环的关键在于：**Thought** 和 **Action** 交替进行，模型每一步都基于上一步的观察结果做决策，而不是一次性把所有动作都规划好再执行。

实现建议

- 在 Function Calling 的 tools 定义中加入 `execute_python(code: str)` 工具
- 工具描述中明确说明：用于精确数值计算，不要用 LLM 自己做算术
- 模型输出代码后，在沙箱中执行（用 `subprocess` 或 `exec`），返回 `stdout/stderr`
- 代码执行超时设为 5 秒，防止死循环
- 如果代码报错，把错误信息返回给模型，让模型自己修

一个额外的收益：**Bash** 执行的 **Token** 消耗远低于让模型一步步推理。一段 Python 代码 50 个 token，执行结果 10 个 token；而同样的计算让模型用 CoT 推理可能需要 200+ token。

7.4 smolagents: 用 Code Agent 范式重新组织工具调用

什么是 smolagents

smolagents (HuggingFace, 2025 年发布, 持续更新中, GitHub: [huggingface/smolagents](#)) [13] 是一个轻量级的 Agent 框架, GitHub 超过 24,000 星标。它的核心设计理念是: 让 **Agent** 通过编写 **Python** 代码来执行操作, 而不是通过 **JSON** 格式的工具调用。

这个区别看似只是“输出格式不同”, 实际上影响了 Agent 的整个推理效率。

传统 Tool Calling vs Code Agent

传统 Function Calling 的流程:

Plaintext

模型推理 → 输出 JSON {"tool": "search", "params": {"query": "退保金"}}}

→ 外部系统解析 JSON → 执行工具 → 返回结果

→ 模型再推理 → 输出 JSON {"tool": "calculator", ...}

→ 外部系统再解析 → ...

每一步都需要 JSON 序列化/反序列化, 且模型只能一次调用一个工具 (或并行的几个工具), 无法写条件判断、循环、中间变量等编程逻辑。

Code Agent 的流程:

模型推理后直接写一段 Python 代码:

Python

```
# 第一步: 搜索退保条款
```

```
results = search("退保金 计算公式")
```

```
# 第二步: 提取数值并计算
```

```
premium = 100000 # 从搜索结果中提取
```

```
interest = 20000
```

```
cash_value = premium + interest * 0.75
```

```
# 第三步: 输出结果
```

```
final_answer(f"退保金为 {cash_value} 元")
```

代码一次性执行完毕, 中间结果自动传递, 不需要反复序列化。

效率提升

据 HuggingFace 的实验数据，Code Agent 相比 JSON Tool Calling 减少了约 30% 的推理步骤。原因是：

1. **批量操作**：一次代码块可以包含多个工具调用，不需要"调一个 → 等结果 → 再调下一个"
2. **变量传递**：中间结果存为 Python 变量，自动流入下一步，不需要 JSON 来回传
3. **条件逻辑**：可以用 `if/else` 做分支判断，比如"如果搜索结果不相关，换个关键词重搜"
4. **模型原生能力**：LLM 在代码生成上的训练数据远多于 JSON 工具调用格式，生成质量更稳定

对本赛题的适用性

smolagents 天然适合本赛题的几个场景：

公式计算题：直接写 Python 代码做数值计算，精确无误。

跨文档对比题：一次代码块中调用多次 `search` 工具（分别搜不同文档），收集所有数据后再统一计算。

多步推理题：用 `if/else` 做条件分支。比如"如果资产负债率超过 70%，触发特别决议；否则普通决议"。

自检题：代码中可以加入断言或验证逻辑。比如 `assert result > 0`，"计算结果不应为负"。

进阶 baseline 集成方案

在 `advance-baseline/` 下新建一个基于 `smolagents` 的进阶版本，架构如下：

```
Plaintext
advance-baseline/
├── src/
│   ├── __init__.py
│   └── agent/
│       ├── __init__.py
│       ├── smol_agent.py      # smolagents CodeAgent 主逻辑
│       └── tools.py          # 自定义工具：文档搜索、BM25 检索、
Python 执行
│   ├── data/
│   │   ├── __init__.py
│   │   └── loader.py         # 题目加载 + 文档读取（复用 baseline
逻辑）
│   ├── evaluation/
│   │   ├── __init__.py
│   │   └── evaluator.py      # 答案校验与格式化
│   └── utils/
│       ├── __init__.py
│       ├── config.py
│       └── llm_client.py     # LLM 客户端（复用 baseline 逻辑）
├── config.yaml
├── requirements.txt
└── run_agent.py              # 入口
```

关键设计决策：

1. **自定义工具集**：注册 `search_document(query, doc_id)` 和 `execute_python(code)` 两个核心工具，`smolagents` 的 `@tool` 装饰器让工具定义极其简洁
2. **模型无关**：`smolagents` 支持百炼（OpenAI 兼容 API）和阶跃星辰，通过 `LiteLLM` 适配，环境变量切换即可
3. **安全执行**：Python 代码在受限的 `LocalPythonInterpreter` 中执行，限制 `import os/subprocess` 等危险模块
4. **Token 预算控制**：`smolagents` 本身不额外消耗 Token，但 `Code Agent` 的输出比 `JSON Tool Calling` 更紧凑

与 **baseline** 的对比

维度	baseline (no-tool)	smolagents 进阶版
文档处理	硬截断前 4000 字符	BM25 检索 + Code Agent 自主决定搜索策略
计算能力	LLM 直接推理（易出错）	Python 代码执行（精确）
推理模式	单轮	ReAct 多轮循环
跨文档	所有文档拼接截断	Code Agent 逐份搜索、聚合计算
Token 效率	固定消耗	按需搜索，代码执行不消耗 LLM Token
实现复杂度	低	中（需要理解 smolagents 框架）

附录 A：技术概念速查

本附录面向第一次接触 Agent、RAG、Token 计算等技术概念的参赛者。如果教程正文中提到的某个术语让你感到陌生，先来这里查找。

A.1 什么是 Agent（智能体）

Agent 是指大语言模型不再只做“你问我答”的被动响应，而是能够主动观察环境、做出决策、调用工具、并根据反馈调整下一步行动的系统。

用一个比喻说明区别：

- **单次调用**：你问“今天北京天气怎么样”，模型直接回答。这是“一问一答”，没有后续动作。
- **工作流（Workflow）**：你让模型“先查天气，再推荐穿衣”，但每一步都是你提前写死的，模型只是按顺序执行。比如第一步固定调用天气 API，第二步固定调用穿衣推荐模板。
- **Agent**：你给模型一个目标“帮我规划一次北京三日游”，模型自己决定：第一步查天气，第二步搜景点，第三步查交通，第四步汇总成行程表。如果查到的景点因天气原因不适合，模型会自己回到第一步重新查，而不是死板地继续执行原计划。

本赛题叫“金融长文本 Agent 的动态记忆压缩与高效问答挑战”，这里的 Agent 强调的是模型在读取长文档的过程中，能够动态决定保留哪些信息、压缩哪些信息、什么时候停止阅读。本 baseline 的硬截断方案其实不算真正的 Agent，因为它没有任何决策能力，只是固定截断 4000 字符。

为什么赛题强调 Agent 而不是简单的问答

因为如果只是“把文档塞给模型、让模型选答案”，那和普通的 RAG 没有区别。赛题的难点在于文档太长（一篇年报 10 万字），不能全部塞进去，所以系统必须具备**自主决策能力**：判断哪些内容需要精读、哪些可以跳过、读完一遍后信息是否足够、不够的话再从哪里补。这种“读一点、判断一下、再决定读不读”的循环，才是 Agent 的核心特征。

A.2 Embedding 与向量检索

Embedding（嵌入）是一种把文本转换成数字向量的技术。它的核心思想是：意思相近的文本，在向量空间中的位置也相近。

具体过程：

1. 选一个 Embedding 模型（如 text-embedding-3、bge-large-zh），输入一段文本
2. 模型输出一个固定长度的向量（比如 768 维或 1536 维），这串数字就是这段文本

的"指纹"

3. 两段文本如果语义相近（比如"退保费用"和"退保手续费"），它们的向量指纹也会很接近

向量检索就是把所有文档片段的向量存进数据库（如 Milvus、Qdrant、FAISS），收到问题后，把问题也转成向量，计算它和所有片段向量的相似度，取最相似的前 K 个。

计算相似度最常用的方法是**余弦相似度**：把两个向量看成空间中的两个箭头，夹角越小（方向越一致），相似度越高。余弦相似度的取值范围是 -1 到 1，实际使用时通常只关心 0 到 1 的区间，0.8 以上就算比较相似。

Embedding 检索 vs 关键词搜索

维度	关键词搜索（如 BM25）	Embedding 向量检索
匹配依据	词是否相同	语义是否相近
"退保费用"搜"退保手续费"	搜不到（词不同）	能搜到（语义相近）
"2025 年营收"搜"去年收入"	搜不到	可能搜到
对数字/年份的敏感度	高	低
计算开销	低	高（需要 GPU/专用模型）
是否需要外部模型	否	是

本赛题没有采用 Embedding 检索，原因见 3.2 节混合检索的讨论：官方未明确 Embedding 模型是否可用，以及其 Token 消耗如何计入总预算。

A.3 Rerank（重排序）

Rerank 是在初步检索结果之上做二次精排的技术。

为什么需要 Rerank？因为初步检索（无论是 BM25 还是 Embedding）追求速度，通常用比较轻量的算法，结果中可能混入一些"看起来相关但不精确"的片段。Rerank 用更精细的模型（通常是 Cross-Encoder）对 top K 结果逐一打分，把真正相关的排在前面。

具体流程：

1. **粗排（召回）**：用 BM25 或 Embedding 快速从上万片段中挑出 50-100 个候选
2. **精排（Rerank）**：把"问题 + 每个候选片段"一起送入 Cross-Encoder，模型输出一个相关性分数
3. **取 top K**：按分数排序，只把最高分的 5-10 个片段交给 LLM

Cross-Encoder 的工作原理和 Embedding 不同：Embedding 是分别把问题和片段转成向量再比较；Cross-Encoder 是把问题和片段拼接成一段文字一起输入模型，让模型直接判断"这段文字和问题有多相关"。Cross-Encoder 更精确，但速度更慢，所以只用在少量候选上。

搜索引擎（百度、谷歌、夸克）的搜索结果页其实也是这个逻辑：先通过倒排索引快速召回百万网页，再用复杂的排序算法（考虑了上百个特征）把最相关的排到前面。

A.4 BM25

BM25 是一种基于词频统计的文本检索算法，是信息检索领域最经典、最常用的算法之一。它的核心思想很简单：如果一个词在文档中出现次数多、在整体文档集中出现次数少，那么这个词对该文档的区分度就高。

BM25 计算每个词对文档的相关性贡献：

- **词频（TF）**：词在文档中出现的次数。出现越多，贡献越大。但不是线性增长（出现 100 次不代表比出现 10 次重要 10 倍），BM25 用一个对数函数做饱和处理。
- **逆文档频率（IDF）**：词在所有文档中出现的频繁程度。"的""是"这种词在每篇文档都出现，IDF 很低；"退保费用"只在保险条款中出现，IDF 很高。

最终得分 = 所有查询词的 TF × IDF 之和。

BM25 的优点：

- 纯统计方法，不依赖任何外部模型，计算速度快
- 对精确匹配（如数字、年份、条款编号）很敏感
- 实现简单，jieba 分词 + rank-bm25 库几行代码就能跑

BM25 的缺点：

- 无法理解语义。"退保费用"和"退保手续费"会被当成完全不同的词
- 对同义词、近义词无能为力
- 长文档天然占优（词频容易更高），需要做长度归一化

A.5 Token 与上下文窗口

Token 是大语言模型处理文本的基本单位。它不是字符，也不是单词，而是模型在训练时从海量文本中学到的"词片段"。

中英文的 Token 换算大致如下：

- 1 个英文单词 $\approx$ 1-2 个 Token
- 1 个汉字 $\approx$ 1-1.5 个 Token（具体取决于模型 tokenizer）
- 1 个标点符号 $\approx$ 1 个 Token
- 数字通常逐位拆分为 Token（"2025"可能是 2 个 Token："20"+"25"）

举例："中国平安的 2024 年营业收入是 1 万亿元"这句话，Qwen 系列模型大约需要 15-20 个 Token。

上下文窗口（Context Window） 是指模型一次能处理的 Token 数量上限。Qwen-plus 从 Qwen-Plus-2025-07-28 快照版本起支持 1M Token（约 80 万汉字），默认 128K（约 10 万汉字），可通过 `max_input_token` 参数调整 [27]。看起来很长，但实际使用时有两个约束：

1. **经济约束**：prompt 越长，消耗的 Token 越多，费用越高。本赛题 500 万 Token 的总预算，意味着你全部用完的话，平均每条 prompt 只能花 5 万个 Token（100 题）。如果把 10 万字的年报全文塞进去，一道题就花掉 10 万 Token，100 题就超预算了。
2. **注意力衰减**：即使模型技术上支持 128K 上下文，实验表明模型对 prompt 中间部分的内容记忆效果会下降（称为"中间丢失"问题）。关键信息如果藏在 prompt 中段，模型可能"视而不见"。

这就是本赛题"动态记忆压缩"的核心挑战：不能无脑塞全文，必须在"保留足够信息"和"控制 Token 消耗"之间做取舍。

附录 B：五类金融文档背景速查

本附录介绍赛题涉及的五种金融文档类型，帮助没有金融背景的参赛者快速理解这些文档的结构和阅读重点。文档类型不同，题目考查的角度也不同，理解文档结构有助于设计更有针对性的检索和阅读策略。

B.1 保险条款

保险条款是保险公司与被保险人之间的合同文本，具有法律效力。条款的措辞非常精确，任何模糊表述都可能导致理赔纠纷。

典型结构：

1. **保险责任**：保什么。比如"身故保险金""重大疾病保险金""住院医疗费用"
2. **责任免除**：不保什么。通常用"因下列情形之一导致...的，本公司不承担保险责任"开头。这是条款定位题的高频考点。
3. **保险期间与等待期**：等待期内出险通常不赔或只退保费，但意外伤害导致的出险往往例外（"因意外伤害发生上述情形的，无等待期"）。
4. **保险金额与保费**：基本保额、累计保费、个人账户价值之间的关系。
5. **退保与现金价值**：退保能拿回多少钱，通常和已交保费、持有年限、账户收益挂钩。不同产品的现金价值计算公式差异很大。
6. **释义**：对条款中出现的专业术语（如"意外伤害""重大疾病""医院"）给出明确定义。

答题关键：

- 计算题要精准定位公式和参数位置，注意年限对应的系数
- 等待期题要找"意外伤害"例外条款
- 免责范围题要到对应条款的"责任免除"章节逐条核对
- 跨产品比较题需要同时打开多份条款，提取同类字段

B.2 监管法规

监管法规是金融监管机构发布的规范性文件，具有强制约束力。参赛涉及的法规主要包括反洗钱、客户尽职调查、受益所有人识别等方面。

典型结构：

1. **总则/适用范围**：法规适用的主体、业务类型、地域范围
2. **具体义务**：金融机构需要做什么。比如"应当识别受益所有人""应当保存客户身份资料至少 5 年"

3. **时限要求**：很多条款有明确的时间节点（"在建立业务关系后 10 个工作日内""在获知变更信息后 30 日内"）
4. **例外条款**："但书"（"但是..."）后面的内容往往是考点。比如"应当识别受益所有人，但各级党的机关除外"
5. **罚则/法律责任**：违反规定会受到什么处罚

答题关键：

- 法条引用题要找原文出处，不能凭印象判断
- 时限题要精确定位具体天数/工作日
- 适用范围题要注意"除外"条款
- 不同法规之间可能存在冲突或补充关系，需要判断优先适用哪一条

B.3 财务报表

财务报表反映企业的经营成果、财务状况和现金流量。赛题涉及的主要是上市公司年报中的三大报表。

三大报表：

1. 资产负债表 (Balance Sheet)

- 反映某一时点的财务状况
- 基本等式：资产 = 负债 + 所有者权益
- 关键科目：货币资金、应收账款、存货、固定资产、短期借款、长期借款、股东权益

2. 利润表 (Income Statement)

- 反映一段时间内的经营成果
- 基本等式：营业收入 - 营业成本 - 费用 = 净利润
- 关键科目：营业总收入、营业成本、销售费用、管理费用、研发费用、营业利润、净利润、归属于母公司股东的净利润

3. 现金流量表 (Cash Flow Statement)

- 反映一段时间内的现金流入和流出
- 分三类：经营活动、投资活动、筹资活动
- 关键指标：经营活动产生的现金流量净额（衡量企业"造血能力"的核心指标）

常用财务指标：

指标	公式	含义
毛利率	$(\text{营业收入} - \text{营业成本}) / \text{营业收入}$	产品本身的盈利能力
净利率	$\text{净利润} / \text{营业收入}$	最终盈利能力
ROE（净资产收益率）	$\text{净利润} / \text{平均股东权益}$	股东投入资本的回报
研发投入占比	$\text{研发费用} / \text{营业收入}$	企业对研发的重视程度

答题关键：

- 跨年度对比题要从两年报表中提取同一指标
- 跨公司对比题要注意指标口径是否一致
- 增长率计算要确认基期和报告期
- 注意"归属于母公司股东的净利润"和"净利润"的区别（前者扣除了少数股东损益）

B.4 金融合同

赛题中的金融合同主要是债券募集说明书和债券条款。

典型结构：

1. **发行概况：** 发行人、债券类型、发行规模、期限、利率
2. **募集资金用途：** 资金投向、使用计划
3. **评级信息：** 主体评级、债项评级、评级展望
4. **增信措施：** 担保、抵押、质押等信用增进安排
5. **特殊条款：**
 - 回售条款：投资者有权在特定时间按约定价格将债券卖回给发行人
 - 赎回条款：发行人有权在特定时间提前赎回债券
 - 调整票面利率条款：发行人有权调整后续计息期间的利率
 - 交叉违约条款：发行人在其他债务上违约，本债券也视为违约

答题关键：

- 条款判断题要定位到具体条款原文，不能凭常识
- 计算题（如回售收益率）要确认时间节点和利率
- 评级相关题要区分"主体评级"和"债项评级"

B.5 行业研报

行业研报是券商研究机构发布的对某个行业的深度分析，通常包含数据、观点、预测和图表。

典型结构：

1. **行业概况：** 行业定义、产业链、市场规模
2. **竞争格局：** 主要参与者、市场份额、集中度（CR4、CR8）
3. **驱动因素：** 政策、技术、需求、供给等影响行业发展的关键因素
4. **数据与预测：** 历史数据、增长率、未来预测
5. **投资建议：** 看好/看平/看空，风险提示

答题关键：

- 事实查找题要注意年份限定（"2022 年"和"2023 年"数据可能完全不同）
- 研报中的数据通常有明确来源标注，但模型可能混淆不同来源的数据
- 研报观点（"我们认为..."）和数据事实要区分开，观点不一定正确

参考文献

- [1] Marker: Convert PDF to Markdown Quickly with High Accuracy. VikParuchuri, 2024-2025. GitHub: [VikParuchuri/marker](https://github.com/VikParuchuri/marker)
- [2] MinerU: An Open-source Solution for Precise Document Content Extraction. Wang et al., 2024. arXiv:2409.18839. 持续更新至 2026 年 4 月 v3.1.0。GitHub: [opendatalab/MinerU](https://github.com/opendatalab/MinerU)
- [3] OmniDocBench: Benchmarking Diverse PDF Document Parsing with Comprehensive Annotations. Ouyang et al., 2024. arXiv:2412.07626
- [4] olmOCR: Unlocking Trillions of Tokens in PDFs with Vision Language Models. Poznanski et al. (Allen AI), 2025. arXiv:2502.18443
- [5] From Local to Global: A Graph RAG Approach to Query-Focused Summarization. Edge et al. (Microsoft Research), 2024. arXiv:2404.16130
- [6] LightRAG: Simple and Fast Retrieval-Augmented Generation. Guo et al. (香港大学 HKUDS 实验室), 2024. arXiv:2410.05779. EMNLP 2025
- [7] StructRAG: Boosting Knowledge Intensive Reasoning of LLMs via Inference-time Hybrid Information Structurization. Li et al., 2024. arXiv:2410.08815
- [8] RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval. Sarthi et al., 2024. arXiv:2401.18059
- [9] Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection. Asai et al., 2024. arXiv:2310.11511
- [10] Corrective Retrieval Augmented Generation. Yan et al., 2024. arXiv:2401.15884
- [11] Precise Zero-Shot Dense Retrieval without Relevance Labels (HyDE). Gao et al., 2023. arXiv:2212.10496
- [12] Searching for Best Practices in Retrieval-Augmented Generation. Wang et al., 2024. arXiv:2407.01219
- [13] smolagents: Minimalist Agent Framework. HuggingFace, 2025. GitHub: [huggingface/smolagents](https://github.com/huggingface/smolagents)
- [14] KAG: Boosting LLMs in Professional Domains via Knowledge Augmented Generation. Liang et al., 2024. arXiv:2409.13731
- [15] ColPali: Efficient Document Retrieval with Vision Language Models. Faysse et al., 2024. arXiv:2407.01449. 以及 ColQwen: Visual Document Retrieval with Qwen2-VL, 2025
- [16] MemGPT: Towards LLMs as Operating Systems. Packer et al. (UC Berkeley), 2023. arXiv:2310.08560. 以及 MemoryOS (2025)、A-Mem (2025) 等后续工作
- [17] ReAct: Synergizing Reasoning and Acting in Language Models. Yao et al., 2023. ICLR 2023. arXiv:2210.03629
- [18] Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. Lewis et

al., 2020. NeurIPS 2020. arXiv:2005.11401

[19] Kimi Code CLI. Moonshot AI, 2025. GitHub: [moonshotai/kimi-code](#)

[20] Hermes Agent Framework. GitHub: [antgroup/hermes](#)

[21] Claude Code. Anthropic, 2025. <https://docs.anthropic.com/en/docs/claude-code>

[22] Codex. OpenAI, 2025. GitHub: [openai/codex](#)

[23] CheetahClaws Agent Framework. GitHub: [cheetahclaws/cheetahclaws](#)

[24] generic-agent: Long-Term Autonomous Agent Architecture. GitHub: [generic-agent](#)

[25] MemoryOS: A Memory Operating System for LLM Agents. 2025.

[26] A-Mem: Agentic Memory for LLM Agents. 2025.

[27] 阿里云百炼. Qwen-Plus 模型更新公告 (2025 年 8 月 1 日, 上下文长度扩展至 1M) . <https://www.aliyun.com/notice/detail?notice-id=117430>